NEU – DS&AI

Data Structures and Algorithms in Python
Lecture 4: Asymptotic Analysis and Big-O Notation

Faculty of Data Science and Artificial Intelligence

National Economics University

Lecture 4: Table of Contents

FDA

SECTION 1

Motivation & History: Why Asymptotic Analysis?

The Limits of Exact Operation Counting

Limitations of exact counting

- **Hardware diversity:** Operations take different CPU cycles ($+$ vs $\div$ vs Memory Access).
- **Language & Compiler:** Python bytecode $\neq$ C machine code $\neq$ Java JIT.
- **Tedious math:** Finding exact constants (e.g., $T(n) = 4n^2 + 7n + 15$) offers minimal practical value.

The Asymptotic Principle

Instead of asking:

"How many microseconds on machine X?"

We ask:

"How does time scale as input size $n \to \infty$?"

Asymptotic analysis focuses on the rate of growth while suppressing hardware-specific constants.

FDA

Why Growth Rate Beats Hardware Speed

Core Takeaway

A fast algorithm on a slow machine will always beat a slow algorithm on a supercomputer for large n!

Algorithm	Growth Type	Hardware	Time for $n = 10^7$
Alg A $(100n)$	Linear (n)	Slow PC (10^6 ops/s)	**1,000 s** (≈ 16.7 min)
Alg B $(0.01n^2)$	Quadratic (n^2)	Supercomputer (10^9 ops/s)	**1,000,000 s** (≈ 11.5 days)
Alg C (2^n)	Exponential (2^n)	Quantum Computer	$> 10^{100}$ **years**

Rule 1: Drop Low-Order Terms

As $n \to \infty$, $n^2 + 1000n \approx n^2$ ($1000n$ becomes insignificant).

Rule 2: Drop Constant Factors

$100n$ and $2n$ belong to the same linear growth class ($t \propto n$).

FDA

A Brief History of Asymptotic Notations

1894: Bachmann

Paul Bachmann
Analytische Zahlentheorie

Introduced the **Big-**O symbol (standing for *Ordnung*, meaning "order of").

1909: Landau

Edmund Landau
Handbuch der Lehre...

Adopted and popularized O and o across mathematics (*Bachmann–Landau notations*).

1976: Knuth

Donald E. Knuth
SIGACT News (1976)

Formalized $\mathcal{O}$, Ω, and Θ for **Computer Science** to standardize algorithm analysis.

Donald Knuth's Insight (1976)

"...we should define $\mathcal{O}, \Omega, \Theta$ to make asymptotic analysis precise and universally understood in Computer Science."

FDA

SECTION 2

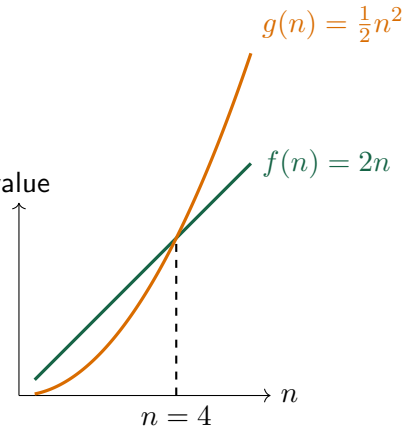
Big O, Big Ω, and Big Θ Notations

Comparing two functions on $\mathbb{N}$

Which one is bigger?

$f(n) = 2n$ or $g(n) = \frac{1}{2}n^2$.

n	1	2	4	10	10^2	10^3	10^9
$g(n)$	0.5	2	8	50	5×10^3	5×10^5	5×10^{17}
$f(n)$	2	4	8	20	2×10^2	2×10^3	2×10^9



- $f(n) \geq g(n)$ for $n \leq 4$
- $f(n) < g(n)$ for all $n > 4$

$g(n)$ becomes much bigger than $f(n)$ when $n \to \infty$

$f(n) < g(n)$ for all sufficiently large n

FDA

Comparing two functions on $\mathbb{N}$

Can you give more examples of function $f(n)$ such that $f(n) < \frac{1}{2}n^2$?

Determine **one possible** $n_0 \in \mathbb{N}$ such that

- $f(n) = n \log n < \frac{1}{2}n^2$ for $n > n_0$
- $f(n) = n + 1 < \frac{1}{2}n^2$ for $n > n_0$
- $f(n) = \sqrt{n} < \frac{1}{2}n^2$ for $n > n_0$
- $f(n) = \log n < \frac{1}{2}n^2$ for $n > n_0$
- $f(n) = 3 < \frac{1}{2}n^2$ for $n > n_0$

Note: Many choices may work. Any valid threshold is acceptable; it does not have to be the smallest.

FDA

More examples of function comparison

$n \log n$ versus $n\sqrt{n}$

n	$n \log n$	$n\sqrt{n}$
16	64	64
64	384	512
256	2,048	4,096

Question?

Can you prove that, for $n > 16$

$$n \log n < n\sqrt{n}?$$

2^n versus $n!$

n	2^n	$n!$
4	16	24
8	256	40,320
12	4,096	479,001,600

Question?

Can you prove that, for $n \geq 4$:

$$2^n < n!?$$

FDA

Big O-Notation: upper bound

Definition

For a given function $g(n)$,

$$O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 \text{ such that } 0 \leq f(n) \leq c\, g(n),\ \forall n \geq n_0\}.$$

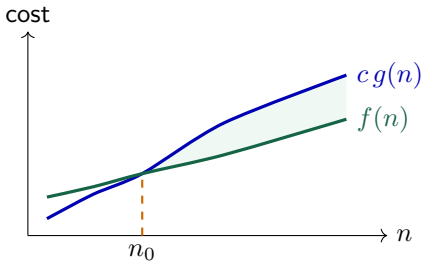
Important: $O(g(n))$ is a **set/class of functions**, not one function. The pair (c, n_0) can be **any valid pair**; it need not be smallest in any metric.

Interpretation

$O(g(n))$ is the class of functions that are eventually **not larger than** a constant multiple of $g(n)$.

Examples

- $2n^2 + 4n + 1 \in O(n^2)$
- $n \log n, n + 1, \sqrt{n}, \log n, 3 \in O(n^2)$
- $n^3, 2^n \notin O(n^2)$



FDA

Algorithmic Intuition of Big-O ($\leq$ Upper Bound)

What does $f(n) = \mathcal{O}(g(n))$ mean for Algorithms?

An algorithm with growth $f(n)$ will run no slower (grow no faster) than $g(n) \times c$ when n is sufficiently large.

- $f(n)$ is **capped from above** by $c \cdot g(n)$ for all $n \geq n_0$.
- Represents a **worst-case upper bound** (guarantees execution time ceiling).

Fast Growth Classes ($\leq$ Linear)

- $\mathcal{O}(1)$: Indexing `A[i]`, Stack `push/pop`
- $\mathcal{O}(\log n)$: **Binary Search**, Euclid GCD
- $\mathcal{O}(n)$: **Linear Search**, find Max/Min

Higher Growth Classes

- $\mathcal{O}(n \log n)$: **Merge Sort**, **Heap Sort**, **Quick Sort**
- $\mathcal{O}(n^2)$: **Bubble Sort**, **Selection Sort**
- $\mathcal{O}(2^n)$: Naive Recursive Fibonacci, TSP

FDA

Quick check: Big O

Exercise

For each statement, decide whether it is true or false. If true, give possible constants c and n_0. If false, explain which term grows too fast.

	Statement	**True/False**
1	$4n + 20 \in O(n)$	
2	$3n^2 + 10n + 7 \in O(n^2)$	
3	$n^2 + 100n \in O(n)$	
4	$n \log_2 n + 50n \in O(n \log n)$	
5	$2^n \in O(n^3)$	

FDA

Big O-notation Properties

Properties

- $O(c \cdot f(n)) = O(f(n))$ for any constant $c > 0$.

  Example: $O(4n^2) = O(\frac{1}{4}n^2) = O(n^2)$.

- If $f(n) \in O(g(n))$ and $h(n) \in O(k(n))$, then $f(n) + h(n) \in O(\max\{g(n), k(n)\})$.

  Example: if $f(n) \in O(n^2)$ and $h(n) \in O(n^3)$, then $f(n) + h(n) \in O(\max\{n^2, n^3\}) = O(n^3)$.

- If $f(n) \in O(g(n))$ and $h(n) \in O(k(n))$, then $f(n) \cdot h(n) \in O(g(n) \cdot k(n))$.

  Example: if $f(n) \in O(n^2)$ and $h(n) \in O(n^3)$, then $f(n) \cdot h(n) \in O(n^5)$.

FDA

Example

Prove that:

$$O(5n^3 + 4n^2 + 5n + 7) = O(n^3)$$

Big O-Notation

Example

Prove that:

$$O(5n^3 + 4n^2 + 5n + 7) = O(n^3)$$

Note

For every polynomial of degree $p \geq 1$ with leading coefficient $a_p > 0$:

$$O(a_p n^p + a_{p-1} n^{p-1} + \cdots + a_1 n + a_0) = O(n^p)$$

FDA

Big Ω-Notation: lower bound

Definition

For a given function $g(n)$,

$$\Omega(g(n)) = \{f(n) : \exists c > 0, \exists n_0 \text{ such that } 0 \leq c\, g(n) \leq f(n),\ \forall n \geq n_0\}.$$

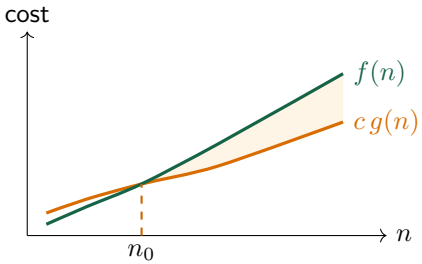
Important: $\Omega(g(n))$ is a **set/class of functions**, not one function. The pair (c, n_0) can be **any valid pair**; it need not be smallest in any metric.

Interpretation

$\Omega(g(n))$ is the class of functions that are eventually **at least as large as** a constant multiple of $g(n)$.

Examples

- $2^n, n^2, n \log n, n + 1 \in \Omega(n)$
- $\frac{1}{2}n - 1 \in \Omega(n)$
- $\sqrt{n}, \log n, 3 \notin \Omega(n)$



FDA

Algorithmic Intuition of Big-Ω ($\geq$ Lower Bound)

What does $f(n) = \Omega(g(n))$ mean for Algorithms?

An algorithm with growth $f(n)$ will take at least $g(n) \times c$ operations when n is sufficiently large (cannot run faster than this bound).

- $f(n)$ is **bounded from below** by $c \cdot g(n)$ for all $n \geq n_0$.
- Proves the **minimum computational effort** required by an algorithm or problem.

Algorithm Lower Bounds

- $\Omega(1)$: Best case of Linear Search
- $\Omega(n)$: Best case of Insertion Sort
- $\Omega(n^2)$: Worst case of Bubble Sort

Problem Lower Bounds

- Finding Min in unsorted array is $\Omega(n)$
- Comparison-based Sorting is $\Omega(n \log n)$
- Matrix multiplication is $\Omega(n^2)$

FDA

Quick check: Big Ω

Exercise

For each statement, decide whether it is true or false. Focus on whether the left-hand function eventually grows at least as fast as the right-hand growth class.

	Statement	**True/False**
1	$7n + 3 \in \Omega(n)$	
2	$n^2 + 5n \in \Omega(n^2)$	
3	$\log n \in \Omega(n)$	
4	$2^n \in \Omega(n^3)$	
5	$n \log n \in \Omega(n^2)$	

FDA

Big Θ-Notation: tight bound

Definition

For a given function $g(n)$,

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2 > 0, \exists n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n),\ \forall n \geq n_0\}.$$

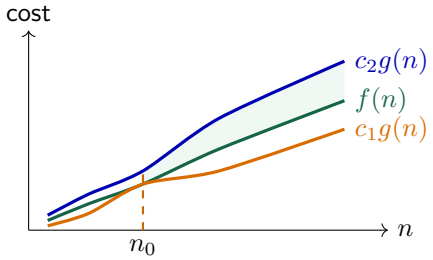
Important: $\Theta(g(n))$ is a **set/class of functions**, not one function. The triple (c_1, c_2, n_0) can be **any valid choice**; it need not be smallest in any metric.

Interpretation

$\Theta(g(n))$ means $f(n)$ is eventually squeezed between two constant multiples of $g(n)$. It is a **tight** growth bound.

Examples

- $n + 1, 2n - 1, \frac{1}{2}n + 3 \in \Theta(n)$
- $n^2, \sqrt{n}, \log n, 3 \notin \Theta(n)$



FDA

Algorithmic Intuition of Big-Θ (= Tight Bound)

What does $f(n) = \Theta(g(n))$ mean for Algorithms?

An algorithm with growth $f(n)$ will grow at the exact same rate as $g(n)$ when n is sufficiently large.

- $f(n)$ is **tightly sandwiched**: $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.
- Holds if and only if $f(n) \in \mathcal{O}(g(n))$ AND $f(n) \in \Omega(g(n))$.

Algorithm Tight Bounds (Θ)

- $\Theta(1)$: Python `len(list)`, `A[i]`
- $\Theta(n)$: Linear Search (worst case)
- $\Theta(n \log n)$: **Merge Sort** (all cases)

Quadratic / Cubic Bounds (Θ)

- $\Theta(n^2)$: **Selection Sort** (all cases)
- $\Theta(n^2)$: Insertion Sort (worst case)
- $\Theta(n^3)$: Floyd-Warshall shortest paths

FDA

Quick check: Big Θ

Exercise

For each running-time function, decide whether it is $\Theta(g(n))$ for the given growth class. Explain using both an upper and a lower bound.

	Function $f(n)$	**Claim**	**True/False**
1	$5n + 8$	$\Theta(n)$	
2	$3n^2 + n \log n$	$\Theta(n^2)$	
3	$n^2 + 100n$	$\Theta(n)$	
4	$7 \log n + 2$	$\Theta(\log n)$	
5	$2^n + n^{10}$	$\Theta(n^{10})$	

FDA

Properties of Big Θ and Ω Notations

Properties

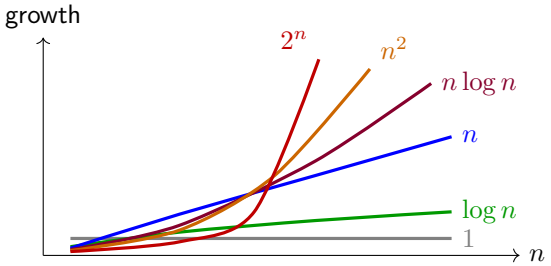
- $\Omega(c \cdot f(n)) = \Omega(f(n))$ for any constant $c > 0$
- If $f(n) \in \Omega(g(n))$ and $h(n) \in \Omega(k(n))$, then $f(n) + h(n) \in \Omega(\max\{g(n), k(n)\})$
- If $f(n) \in \Omega(g(n))$ and $h(n) \in \Omega(k(n))$, then $f(n) \cdot h(n) \in \Omega(g(n) \cdot k(n))$.

These properties also hold for Θ notation.

FDA

Common function classes

1	$\log n$	n	$n \log n$	n^2	2^n
constant	logarithmic	linear	quasi-linear	quadratic	exponential



FDA

Intuitive Meaning of $\mathcal{O}$, Ω, and Θ at a Glance

$\mathcal{O}(g(n))$: Upper Bound

Analogy: $\leq$ (*No slower than*)

$f(n) \leq c \cdot g(n)$

Meaning: Cost grows **at most as fast as** $c \cdot g(n)$ as $n \to \infty$.

Guarantees worst-case ceiling.

$\Omega(g(n))$: Lower Bound

Analogy: $\geq$ (*At least as slow as*)

$f(n) \geq c \cdot g(n)$

Meaning: Cost is **at least** $c \cdot g(n)$ for large n.

Proves problem hardness / baseline.

$\Theta(g(n))$: Tight Bound

Analogy: $\approx$ (*Same rate as*)

$c_1 g(n) \leq f(n) \leq c_2 g(n)$

Meaning: Cost grows at the **exact same rate**.

Both $\mathcal{O}(g(n))$ and $\Omega(g(n))$ hold.

Summary for Programmers

Saying $f(n) = \mathcal{O}(g(n))$ **means:** For large input size n, the algorithm with cost $f(n)$ will **never run slower** than $g(n) \times$ constant.

FDA

Classic Named Algorithms by Complexity Class

Class	Name	Representative Algorithms / Operations	Practical Scalability
$\mathcal{O}(1)$	Constant	Array index `A[i]`, Stack `push`/`pop`, Hash map	Instantaneous ($< 1\ \mu$s)
$\mathcal{O}(\log n)$	Logarithmic	**Binary Search**, Balanced BST lookup, Euclid GCD	Ultra fast ($n = 10^9 \implies 30$ ops)
$\mathcal{O}(n)$	Linear	**Linear Search**, find Max/Min, List traversal	Scales proportionally with n
$\mathcal{O}(n \log n)$	Quasi-linear	**Merge Sort**, **Heap Sort**, **Quick Sort** (avg)	Optimal for general sorting
$\mathcal{O}(n^2)$	Quadratic	**Bubble Sort**, **Selection Sort**, **Insertion Sort**	Feasible for $n \leq 10^4$
$\mathcal{O}(n^3)$	Cubic	Matrix Multiplication, **Floyd-Warshall**	Feasible for $n \leq 10^3$
$\mathcal{O}(2^n)$	Exponential	Naive Fibonacci, Subset Sum (brute force)	Impractical for $n > 40$
$\mathcal{O}(n!)$	Factorial	Generating permutations, **TSP** (brute force)	Impractical for $n > 12$

FDA

Understanding Ω and Θ with Real Algorithm Examples

Lower Bound Ω Examples

- **Finding Minimum is $\Omega(n)$:** Must inspect every element in an unsorted array $\implies$ no algorithm can beat $\Omega(n)$.
- **Comparison Sorting is $\Omega(n \log n)$:** Any comparison sort (MergeSort, QuickSort) must make $\geq \Omega(n \log n)$ comparisons in the worst case.
- **Adjacent-Swap Sorts are $\Omega(n^2)$:** Any sort swapping only adjacent elements requires $\Omega(n^2)$ in the worst case.

Tight Bound Θ Examples

- **Merge Sort is $\Theta(n \log n)$:** Always splits in half and merges $\implies$ $\Theta(n \log n)$ in **Best, Worst, and Average** cases!
- **Selection Sort is $\Theta(n^2)$:** Always scans the rest of the array $\implies$ $\Theta(n^2)$ in **all cases**, even if already sorted.
- **Linear Search (Worst-Case) is $\Theta(n)$:** Traversing all n elements takes exact linear time.
- **Python `len(list)` is $\Theta(1)$:** Stored as a struct field, always constant time.

FDA

Example

Question

For each function, identify the dominant term, remove its constant coefficient, and write the simplest Big-O bound.

	Running-time function	**Big-O**
1	$T(n) = 25$	_____
2	$T(n) = 8n + 40$	_____
3	$T(n) = 5n \log_2 n + 2n$	_____
4	$T(n) = 12n^2 + 3n + 7$	_____
5	$T(n) = n^3 + 50n^2 + 10n$	_____
6	$T(n) = n^2 + n \log_2 n + 1000$	_____

FDA

More practice: simplify $T(n)$

Question

For each function, identify the dominant growth term and write the simplest Big-O bound. Assume $n \geq 2$.

	Running-time function	Big-O
1	$T(n) = 18 \log_2 n + 7$	_____
2	$T(n) = 9n\sqrt{n} + 4n$	_____
3	$T(n) = \sum_{i=1}^{n} (3i + 2)$	_____
4	$T(n) = 2n^2 \log_2 n + 6n^2 + 1$	_____
5	$T(n) = 5 \cdot 2^n + n^4$	_____
6	$T(n) = n^3 + 3^n$	_____

FDA

How to simplify a running-time function

Big-O rule

Keep the fastest-growing term and ignore constant factors and smaller terms.

Worst-case operation count	**Keep**	**Big-O**
$T(n) = 7$	constant	$O(1)$
$T(n) = 3n + 5$	n	$O(n)$
$T(n) = 2n^2 + n$	n^2	$O(n^2)$

Why ignore constants?

For large n, $3n + 5$ and n grow in the same linear pattern; the factor 3 changes the amount of work, but not the growth class.

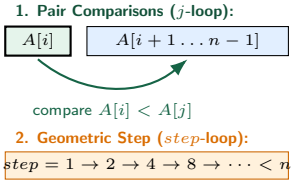
FDA

Pair Scoring Algorithm: Problem & Core Idea

Problem Definition

Given an array `values` of n numbers, compute an aggregated score by performing two independent operations for each element at index i:

1. Compare with every element to its right.
2. Accumulate $A[i]$ in geometric doubling steps $(1, 2, 4, \dots)$.



Loop Invariant

After step i, `score` reflects all valid comparisons and log-accumulations for the entire prefix $0 \dots i$.

Structure Overview

Two sequential inner loops inside an outer loop of size n.

FDA

Activity: Analyze Pair Scoring Algorithm

```
FUNCTION PAIR_SCORE
INPUT: a sequence values of length n
OUTPUT: a numerical score
    score = 0
    FOR i = 0 TO n - 1 DO
        FOR j = i + 1 TO n - 1 DO
            IF values[i] < values[j] THEN
                score = score + 1
        step = 1
        WHILE step < n DO
            score = score + values[i]
            step = 2 * step
    RETURN score
```

Student Task

Assume each basic operation takes $O(1)$ time.

1. **Pair comparisons (j-loop):** How many comparisons occur over all i?
2. **Geometric while-loop:** How many iterations run for each i?
3. **Total operation count $T(n)$:** Combine the two parts into a single function.
4. **Big-O / Big-Θ bound:** State and justify the dominant growth class.

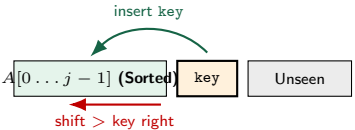
FDA

Insertion Sort: Problem & Core Idea

Problem Definition

Input: An array $A[0 \dots n-1]$ of n elements.
Output: A sorted in non-decreasing order:

$$A[0] \leq A[1] \leq \cdots \leq A[n-1].$$



Core Idea (Card Player Analogy)

- Pick each element `key = A[j]` one by one ($j = 1 \to n-1$).
- Shift all larger elements in the sorted prefix $A[0 \dots j-1]$ one slot to the right.
- Insert `key` into the created empty slot.

Loop Invariant

At the start of step j, the subarray $A[0 \dots j-1]$ consists of the elements originally in $A[0 \dots j-1]$, but in **sorted order**.

FDA

Activity: Analyze Insertion Sort

```
FUNCTION INSERTION_SORT(A)
INPUT: array A of length n
OUTPUT: A sorted in non-decreasing order
    FOR j = 1 TO LENGTH(A) - 1 DO
        key = A[j]
        i = j - 1
        WHILE i >= 0 AND A[i] > key DO
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
    RETURN A
```

Student Task

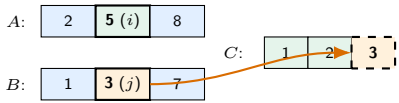
1. **Trace the algorithm:**
   Trace on input $[5, 2, 4, 6, 1, 3]$.
2. **Best-case scenario (Already sorted):**
   Count comparisons and find Big-O bound.
3. **Worst-case scenario (Reverse sorted):**
   Write summation of shifts and find Big-O bound.
4. **Auxiliary space complexity:**
   Determine auxiliary space needed.

FDA

Merge Procedure: Problem & Core Idea

Problem Definition

Input: Two **already sorted** arrays A and B.
Output: A single sorted array C containing all $n = |A| + |B|$ elements in non-decreasing order.



Core Idea (Two-Finger Scan)

- Point i at start of A, j at start of B.
- Compare $A[i]$ and $B[j]$.
- Append smaller item to C and advance that pointer.
- Copy any leftovers when one array is exhausted.

Loop Invariant

$C[0 \ldots i + j - 1]$ always contains the $i + j$ smallest elements of $A \cup B$ in sorted order.

FDA

Activity: Analyze the Merge Procedure

```
FUNCTION MERGE(A, B)
INPUT: sorted arrays A and B
OUTPUT: sorted array C containing all values
    CREATE an empty array C
    i = 0; j = 0
    WHILE i < LENGTH(A) AND j < LENGTH(B) DO
        IF A[i] <= B[j] THEN
            APPEND A[i] to C; i = i + 1
        ELSE
            APPEND B[j] to C; j = j + 1
    WHILE i < LENGTH(A) DO
        APPEND A[i] to C; i = i + 1
    WHILE j < LENGTH(B) DO
        APPEND B[j] to C; j = j + 1
    RETURN C
```

Student Task

Let $n = |A| + |B|$.

1. **Trace the procedure:** Trace for $[2, 5, 8]$ and $[1, 3, 7]$.
2. **Element comparisons:** Maximum comparisons `A[i] <= B[j]`.
3. **Append operations:** Total elements appended into C.
4. **Time complexity:** Find $T(n)$ and its Big-O / Big-Θ bound.
5. **Auxiliary space complexity:** Memory allocated for array C.

FDA

Array Partitioning: Problem & Core Idea

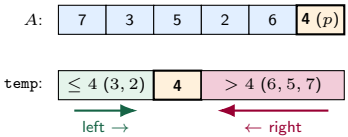
Problem Definition

Input: Subarray $A[low \dots high]$ and pivot $p = A[high]$.
Output: Rearrange elements around p so that elements $\leq p$ are placed left, and elements $> p$ are placed right.

Simplest Idea: Auxiliary Array

Fill a helper array `temp`:

- Items $\leq p \implies$ place at `left` ($\rightarrow$).
- Items $> p \implies$ place at `right` ($\leftarrow$).
- Place p in the middle, then copy back to A.



FDA

Activity: Analyze Quicksort Partition (Auxiliary Array)

```
FUNCTION PARTITION_AUX(A, low, high)
INPUT: subarray A[low..high]
OUTPUT: final index of pivot
    pivot = A[high]
    CREATE array temp of size (high - low + 1)
    left = 0
    right = high - low
    FOR i = low TO high - 1 DO
        IF A[i] <= pivot THEN
            temp[left] = A[i]
            left = left + 1
        ELSE
            temp[right] = A[i]
            right = right - 1
    temp[left] = pivot
    COPY temp BACK TO A[low..high]
    RETURN low + left
```

Student Task

Let subarray size $m = high - low + 1$.

1. **Trace the procedure:**
   Trace on $[7, 3, 5, 2, 6, 4]$ with pivot 4.
2. **Comparisons with pivot:**
   Count comparisons in the `FOR` loop.
3. **Array copy steps:**
   Count items moved into and out of `temp`.
4. **Time complexity:**
   Find $T(m)$ and its Big-O / Big-Θ bound.
5. **Auxiliary space complexity:**
   Memory allocated for array `temp`.

FDA

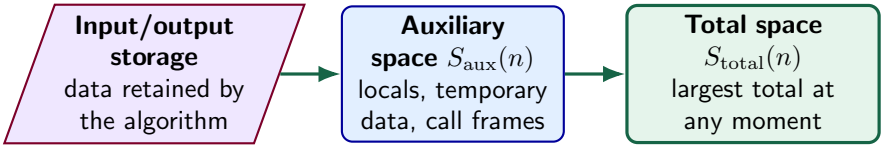
SECTION 3

Space Complexity

Space complexity: a practical view

Definition

Space complexity measures the **maximum memory simultaneously in use** during an algorithm. We study how this worst-case requirement grows as the input size n grows. Write $S_{\text{total}}(n)$ for total space and $S_{\text{aux}}(n)$ for auxiliary space.



Total space: $S_{\text{total}}(n)$

Counts input, output, and temporary working memory that coexist at the peak-memory moment.

Auxiliary space: $S_{\text{aux}}(n)$

Counts only extra working memory, excluding the input and output. Later examples report both measures explicitly.

DA

Examples: calculate $S(n)$

Storage model

Count one unit for each scalar and each array element. Total space includes the input; auxiliary space counts only local working storage.

Squared distance between two points

```
FUNCTION DISTANCE_SQUARED
INPUT: x1, y1, x2, y2
    dx = x2 - x1
    dy = y2 - y1
    answer = dx * dx + dy * dy
    RETURN answer
```

Input: 4 units. Locals: 3 units.

$$S_{\text{total}}(n) = 4 + 3 = 7 \in \Theta(1),$$
$$S_{\text{aux}}(n) = 3 \in \Theta(1).$$

Count positive values

```
FUNCTION COUNT_POSITIVE
INPUT: array values of length n
    count = 0
    i = 0
    WHILE i < n DO
        IF values[i] > 0 THEN
            count = count + 1
        i = i + 1
    RETURN count
```

Input: n units. Locals `count` and `i`: 2 units.

$$S_{\text{total}}(n) = n + 2 \in \Theta(n),$$
$$S_{\text{aux}}(n) = 2 \in \Theta(1).$$

Examples: output storage changes $S(n)$

Convention

Total space includes input and output. Auxiliary space below excludes both and counts only loop variables and temporary working data.

Create a sign mask

```
FUNCTION SIGN_MASK
INPUT: array values of length n
OUTPUT: Boolean array mask of length n
    FOR i = 0 TO n - 1 DO
        mask[i] = (values[i] >= 0)
    RETURN mask
```

Input: n; output: n; index: 1.

$$S_{\text{total}}(n) = 2n + 1 \in \Theta(n),$$
$$S_{\text{aux}}(n) = 1 \in \Theta(1).$$

Build a comparison table

```
FUNCTION COMPARISON_TABLE
INPUT: array values of length n
OUTPUT: n by n Boolean table
    FOR i = 0 TO n - 1 DO
        FOR j = 0 TO n - 1 DO
            table[i][j] = (values[i] < values[j])
    RETURN table
```

Input: n; output: n^2; indices: 2.

$$S_{\text{total}}(n) = n^2 + n + 2 \in \Theta(n^2),$$
$$S_{\text{aux}}(n) = 2 \in \Theta(1).$$

FDA

Function calls and memory reuse

```
FUNCTION PROCESS_THREE
INPUT: reference to array data of length n
    x = ANALYZE(data)
    y = ANALYZE(data)
    z = ANALYZE(data)
    RETURN x + y + z

FUNCTION ANALYZE(data)
INPUT: reference to array data of length n
    temp = a copy of data
    RETURN temp[0]
```

Each call creates one temporary n-item array. The input itself is shared, because it is passed by reference.

Auxiliary memory over time

Moment	Memory in use
before call 1	c
during call 1	$c + n$
after call 1	c
during call 2	$c + n$
during call 3	$c + n$

Peak space

Here c is fixed storage retained by `PROCESS_THREE`. Only one `temp` exists at a time, so

$$S_{\text{aux}}(n) = n + c \in \Theta(n),$$

not $3n$. A completed call releases its local memory for reuse.

Example: analyze space usage

```
FUNCTION IS_PALINDROME
INPUT: a bit string bits of length n
OUTPUT: TRUE if the bit string reads the same
     backward
    reversed = an empty bit string

    FOR i = n - 1 DOWN TO 0 DO
        APPEND bits[i] TO reversed

    FOR i = 0 TO n - 1 DO
        IF bits[i] != reversed[i] THEN
            RETURN FALSE
    RETURN TRUE
```

Memory analysis

- Input string: n bits.
- Reversed copy: n additional bits.
- Loop index and fixed control data: $O(\log n)$ bits.

Conclusion

The temporary copy dominates, so $S_{\text{aux}}(n) \in \Theta(n)$. Total space also satisfies $S_{\text{total}}(n) \in \Theta(n)$.

Can we use less space?

Compare matching bits from the two ends without constructing a copy. Two indices require $\Theta(\log n)$ bits, or $\Theta(1)$ machine words.

FDA

Key takeaways: algorithms and representations

- **Start with the problem:** clearly state the accepted inputs and required outputs.
- **Define the algorithm:** give a finite, precise sequence of effective steps that produces the result.
- **Check all essential properties:** defined input/output, definiteness, finiteness, effectiveness, correctness, and generality.
- **Choose a representation:** use a flowchart to emphasize control flow or pseudocode to express structured, language-neutral steps.
- **Separate logic from implementation:** one algorithm can be implemented as programs in many languages.
- **Validate systematically:** trace examples and edge cases, detect ambiguity or non-termination, and use counterexamples to expose incorrectness.

FDA

Key takeaways: time and space complexity

- **Fix the input size and cost model:** describe the problem using n and decide which basic operations count as constant-time work.
- **Build $T(n)$:** add consecutive work, account for changing loop bounds, and multiply genuinely nested work.
- **Recognize common patterns:** full scans are linear, nested scans are often quadratic, and repeated doubling or halving is logarithmic.
- **State the case:** best, average, and worst cases describe different inputs having the same size.
- **Classify growth:** $f \in O(g)$ gives an upper bound, $f \in \Omega(g)$ a lower bound, and $f \in \Theta(g)$ a tight bound; constants and lower-order terms do not determine the class.
- **Measure peak memory:** $S_{\text{total}}(n)$ counts total memory, while $S_{\text{aux}}(n)$ counts extra working memory beyond input and output.

FDA